Ques. 1: List any two readability, writability and reliability issues in C Language?

1. Readability Issues

Readability refers to how easily a programmer can read and understand the code.

- **Complex Pointer Syntax:** C allows for highly complex and nested pointer declarations. For example, a declaration like `void (*signal(int sig, void (*func)(int)))(int);` is notoriously difficult to parse mentally and understand at a glance.
- **Heavy Use of Preprocessor Macros:** C relies heavily on `#define` macros for constants and inline functions. Because macros are simply text substitutions performed before compilation, they can obscure the actual logic, lack type safety, and make debugging difficult.

2. Writability Issues

Writability refers to how easily a language can be used to create programs for a chosen problem domain.

- **Lack of Built-in High-Level Data Structures:** C does not have native support for complex data structures like lists, stacks, queues, or hash maps. Programmers must build these from scratch or rely on external libraries, which significantly increases coding effort and time.
- **Manual Memory Management Overhead:** The absence of garbage collection means the programmer is entirely responsible for allocating (`malloc`/`calloc`) and deallocating (`free`) memory. This requires writing extra boilerplate code and constantly tracking resource ownership.

3. Reliability Issues

Reliability refers to a program performing to its specifications under all conditions.

- **No Array Bounds Checking:** C does not perform runtime checks to ensure you are accessing memory within the bounds of an array. Writing past the end of an array leads to buffer overflows, which cause undefined behavior, crashes, and severe security vulnerabilities.
- **Memory Leaks and Dangling Pointers:** Because memory management is manual, human error is common. Forgetting to free allocated memory causes memory leaks (eventually crashing the program by exhausting memory), and accessing memory after it has been freed creates dangling pointers, leading to unpredictable program behavior

Ques. 2: Can a faithful procedure produce results in the calling program? Provide a detailed justification of your answer. **[2 Marks]**

Answer: No.

Justification: To produce results in a calling program, a subprogram must either return a value or cause a side effect.

1. **Definition of a Procedure:** In programming language design, a procedure (unlike a function) does not return a value. It relies entirely on side effects (like modifying global variables or pass-by-reference parameters) to pass results back to the caller.
2. **Definition of "Faithful":** A "faithful" subprogram strictly models a pure mathematical function, meaning it has **zero side effects**. It does not alter any external state or parameters passed to it.

Conclusion: Because a procedure lacks a return value, and being "faithful" strips it of the ability to use side effects, it has no mechanism left to produce or communicate results back to the calling program.

Ques. 3: Give the design issues of coroutine. **[Note:** No marks for writing design issues that are applicable for all three (coroutine, function and procedure).**]** **[2 marks]**

1. **Control transfer mechanism**
   The language must define how control is transferred between coroutines, i.e., whether the transfer is **symmetric** (control can pass to any coroutine) or **asymmetric** (control returns to the caller).

2. **Saving and restoring execution state**
   When a coroutine suspends execution, its **current execution point and local variables must be saved** so that execution can continue from the same point when resumed.

3. **Resume and suspend operations**
   The language must provide mechanisms to **suspend the execution of a coroutine and resume it later**.

4. **Lifetime and storage of coroutine variables**
   The design must determine **how long the local variables of a coroutine remain active and how their storage is maintained** between suspensions.

Ques. 4: Give an example for the pure function based on the referential transparency and no side effects. **[2 Marks]**

```
int add(int x, int y) {
   return x + y;
}
```

This function qualifies as a pure function because it strictly satisfies both required properties:

1. **Referential Transparency:** The function consistently yields the exact same output for the same given inputs (e.g., `add(3, 4)` will always return `7`). The function call could be entirely replaced by its resulting value without altering the program's behavior.
2. **No Side Effects:** The function does not modify any external state, global variables, or the parameters passed to it. It strictly computes a result using only the provided local inputs and does not interact with the outside world (like performing I/O operations or writing to a database).

Ques. 5: Differentiate for loop and while loop. **[1 Mark]**

For Loop	While Loop
Used when the **number of iterations is known** in advance.	Used when the **number of iterations is not known** beforehand.
Initialization, condition, and update are written **in one line**.	Initialization is usually done **before the loop** and update **inside the loop body**.

Ques. 6: Consider the following program fragment: **[2 Ma**

a.
```
x := x + 2;
y := x + y;
x := y - 1;
```

Find the weakest precondition (WP) for the postcondition Q : x > 5

b.
```
if (x ≥ 0)
   y := x + 2
else
   y := x
```

We compute WP **backwards**.

1. Last statement:

   `x := y - 1`

   Substitute in Q:

   `y - 1 > 5`

   `y > 6`

2. Second statement:

   `y := x + y`

   Substitute `y` with `x + y`:

   `x + y > 6`

3. First statement:

   `x := x + 2`

   Substitute `x` with `x + 2`:

   `(x + 2) + y > 6`

   `x + y + 2 > 6`

   x + y > 4

(b)

Program:

```
if (x ≥ 0)
   y := x + 2
else
   y := x
```

For **if statement WP rule**:

$$WP = (B \wedge WP(S1, Q)) \vee (\neg B \wedge WP(S2, Q))$$

Where

B : **x ≥ 0**

- For `y := x + 2`

  WP = **y = x + 2**
- For `y := x`

  WP = **y = x**

So,

$$WP = (x \geq 0 \wedge y = x + 2) \vee (x < 0 \wedge y = x)$$

✅ **Weakest Precondition:**

$$(x \geq 0 \wedge y = x + 2) \vee (x < 0 \wedge y = x)$$

Ques. 7: Consider the following grammar with the set of non-terminals {S,P,Q} and set of terminals {a, b}.

$$S \rightarrow PQa$$
$$P \rightarrow Qa \mid \epsilon$$
$$Q \rightarrow Pb \mid \epsilon$$

A. Find first and follow of the each non-terminal symbol. **[2 Marks]**

B. Construct the LL(1) parsing table. **[2 Marks]**

C. State whether the given grammar is LL(1) or not. If not, find the number of conflicts in LL(1) parsing table. **[1 Mark]**

Q7 $S \rightarrow PQa$
$P \rightarrow Qa \mid \varepsilon$
$Q \rightarrow Pb \mid \varepsilon$

(A)

	First	Follow
S	{a, b}	{$}
P	{a, b, ε}	{a, b}
Q	{a, b, ε}	{a}

(B)

	a	b
S	S → PQa	S → PQa
P	P → Qa P → ε	P → Qa P → ε
Q	Q → Pb Q → ε	Q → Pb

(C) Not LL(1) because multiple production appear in the same table entry
cells M[P,a], M[P,b], M[Q,a] have 2 entries each

Ques. 8: Consider the following attribute grammar:

S→id=E; {S.ok=(id.type=E.type)}
E→E+id {E.type=type_check(E.type,id.type)}
E→id {E.type=id.typeE.type = id.typeE.type=id.type}

Here, type is a synthesized attribute representing the variable type, and id.type is obtained from the symbol table.

(a). Determine whether this grammar is S-attributed. Justify your answer. **[1 Mark]**

(b). Determine whether this grammar is L-attributed. Justify your answer. **[1 Mark]**

(c). Indicate which parser (top-down / bottom-up) is suitable for evaluating the attributes of this grammar and explain why. **[1 Mark]**

(a) Determine whether this grammar is S-attributed. [1 Mark]

Answer: Yes, the grammar is S-attributed. **Justification:** An attribute grammar is S-attributed if all of its attributes are strictly synthesized. In this grammar, the attributes (`S.ok` and `E.type`) evaluate their values entirely based on the attributes of their child nodes on the right-hand side of the production. There are no inherited attributes (where a child depends on a parent or sibling).

(b) Determine whether this grammar is L-attributed. [1 Mark]

Answer: Yes, the grammar is L-attributed. **Justification:** By definition, every S-attributed grammar is also automatically an L-attributed grammar. Since L-attributed grammars allow all synthesized attributes and only restrict how *inherited* attributes flow (left-to-right), the complete absence of inherited attributes here means it inherently satisfies the L-attributed conditions.

(c) Suitable parser (top-down / bottom-up) and why. [1 Mark]

Answer: Bottom-up parser. **Justification:** There are two main reasons:

1. **Left Recursion:** The grammar contains a left-recursive production (`E -> E + id`). Top-down parsers (like LL parsers) cannot handle left recursion and will fall into an infinite loop, whereas bottom-up parsers (like LR parsers) handle it naturally.
2. **Attribute Evaluation:** Bottom-up parsers are perfectly suited for S-attributed grammars. They can easily evaluate synthesized attributes alongside the reduction steps as they build the parse tree from the leaves up to the root.

Ques 9: Consider the following program fragment:

z := x / y

Suppose the desired postcondition is Q : z * y = x. Using axiomatic semantics, determine the weakest precondition (WP) for the above statement. **[2 Marks]**

Postcondition:

Q : z * y = x

To find the **Weakest Precondition (WP)**, substitute the assigned expression of `z` in the postcondition.

Step:

Q = `z * y = x`

Substitute `z = x / y`

$$(x/y) * y = x$$

This simplifies to

$$x = x$$

This is **true only when y ≠ 0** because division by zero is undefined.

Weakest Precondition

$$WP = y \neq 0$$

✅ **Answer:** The weakest precondition for the statement `z := x / y` with postcondition `z * y = x` is **y ≠ 0**.

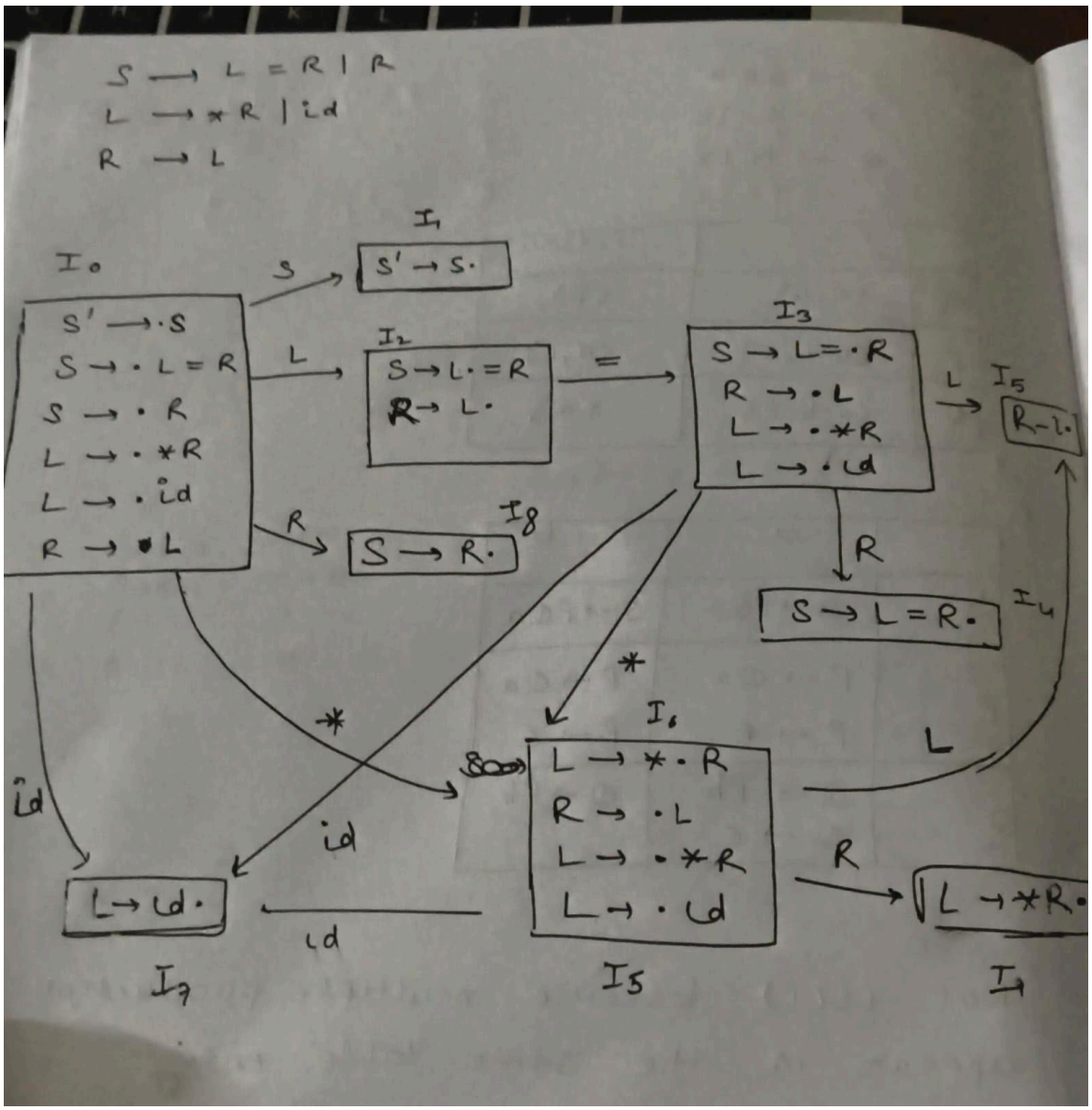
Ques 11: Consider the augmented grammar where {=,*,id} represents the set of terminals.

S→L=R | R
L→*R | id
R→L

a. Check whether the given grammar can be parsed by an LR(0) parser. Justify your answer by constructing the LR(0) items, the canonical collection of sets of LR(0) items, and the LR(0) parsing table. Clearly indicate any conflicts and explain their impact on LR(0) parsing. **[2 Marks]**

b. Check whether the given grammar can be parsed by SLR(1) parser. Provide proper justification. **[1 Marks]**

a)



Conflict

State I_2

S → L • = R
R → L •

- shift on =
- reduce by R → L

So **shift-reduce conflict occurs**.

Conclusion

The grammar **cannot be parsed by LR(0)** because of the **shift-reduce conflict in state I_2**

b)

Non-Terminal	FIRST	FOLLOW
S	$\{*, \mathrm{id}\}$	$\{\$\}$
L	$\{*, \mathrm{id}\}$	$\{=, \$\}$
R	$\{*, \mathrm{id}\}$	$\{=, \$\}$

b. Check whether the given grammar can be parsed by an SLR(1) parser. [1 Mark]

Answer: No, the given grammar cannot be parsed by an SLR(1) parser.

Proper Justification:
An SLR(1) parser attempts to resolve Shift/Reduce conflicts by only performing a Reduce action for a completed item $A \to \alpha\cdot$ when the lookahead symbol is in the $FOLLOW(A)$ set.

Based on the canonical collection of LR(0) items (your DFA), observe **State** I_2:

$$I_2 = \{S \to L\cdot = R, \quad R \to L\cdot\}$$

Let's evaluate the parsing actions for State I_2:

1. **Shift Action:** The item $S \to L\cdot = R$ expects the terminal `=`, so the parser will issue a **Shift** action when the lookahead is `=`.
2. **Reduce Action:** The item $R \to L\cdot$ is a completed item. In SLR(1), we place a **Reduce** action (using the rule $R \to L$) in the parsing table columns corresponding to $FOLLOW(R)$.
3. As calculated in the table above, $FOLLOW(R) = \{=, \$\}$.

The Conflict:
Because the terminal `=` is present in $FOLLOW(R)$, the SLR(1) parser will attempt to place a **Reduce** action in the `=` column for state I_2. However, that same column already requires a **Shift** action from the first item.

Since there are multiple defined actions (Shift and Reduce) for the same state (I_2) and the same lookahead symbol (`=`), a **Shift/Reduce conflict** exists. Thus, the grammar is not SLR(1).